

Interconnection Networks

How processors and memory talk to each other: static and dynamic networks

Lecture 5 (5a + 5b) · Parallel and Distributed Computing

What we will learn today

1. **What a network is**
   and why parallel computers need one
2. **Types of networks**
   by size, and static vs dynamic
3. **Building blocks**
   switches and network interfaces
4. **Topologies**
   bus, crossbar, Omega, complete, star

By the end you can compare topologies on cost and performance.

What is an interconnection network?

- A system that carries data between processors, and between processors and memory
- It is programmable: data can be sent to different places at different times
- Made of switches (decision points) and links (wires or fibre)

Think of a city

Houses are processors and memory. Roads are links. Junctions are switches. Traffic is data.

Example: Computer parts talk through the network the way people travel through a road system.

Better network = less waiting for data = faster programs.

Why not give every part its own wire?

What software wants

Low delay and high speed, dedicated channels between every processor and memory.

What hardware allows

Dedicated channels cost too much chip area and power. Resources are scarce.

Solution: share wires smartly through a network. Managing this communication can decide the speed of the whole program.

Example: Direct wires between all pairs: 8 cores need 28 wires, 100 cores need 4,950 wires.

A network shares scarce resources efficiently.

Design goal of a network

Move more data

Maximum information transferred

In less time

Low delay, high bandwidth

Within limits

Cost, power and area constraints

If the network is too slow, it becomes the bottleneck. Fast processors then sit idle waiting for data.

Example: A fast highway with one narrow bridge: all cars queue at the bridge.

Four kinds of networks by size

On-chip (NoC)

Inside one chip: cores, caches.
Dozens to hundreds of devices.

Ex: Intel 80-core research chip, 48-core Single-chip Cloud Computer

System (SAN)

Supercomputers, data centres, storage.
Hundreds to thousands of devices.

Ex: IBM Blue Gene/L: 64K nodes

Local (LAN)

Machine room, building, campus.
Hundreds of devices.

Ex: Ethernet: 10 Gbps over 40 km

Wide (WAN)

Across the globe.
Millions of devices.

Ex: The Internet, ATM

Network domains at a glance

Domain	Devices	Distance	Example
On-chip (NoC)	Dozens to hundreds	Millimetres	80-core chip
SAN	Hundreds to thousands	Tens to hundreds of metres	Blue Gene/L
LAN	Hundreds (1000s with bridges)	Few km to tens of km	Ethernet
WAN	Millions	Thousands of km	Internet

More devices and more distance means more complexity. This lecture focuses on the first two rows, inside parallel computers.

Static vs dynamic networks

Static (direct)

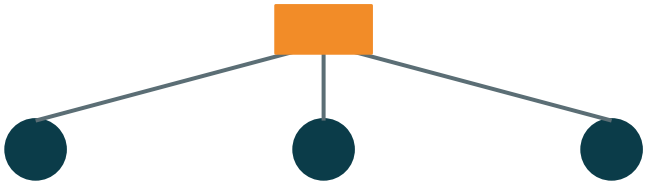
Fixed point-to-point links between processing nodes. No switches in the middle.



Static

Dynamic (indirect)

Built from switches and links. Connections are made when needed.



Dynamic

Example: Static = friends calling each other directly. Dynamic = calls going through a phone exchange.

Switches: the decision makers

- A switch maps a fixed number of inputs to outputs
- Degree of a switch = its total number of ports
- Cost grows as the square of the degree
- Peripheral hardware grows linearly with degree; packaging cost grows with the number of pins

Small switch, cheap

Doubling the ports makes the switch about 4 times as expensive. Many small switches are cheaper than one giant one.

Example: A 4-port switch costs about 16 units; an 8-port switch costs about 64 units.

Network interface

- Processors talk to the network through a network interface
- It can hang off the I/O bus or the memory bus
- This physical difference separates a cluster from a tightly coupled multicomputer
- Relative speed of I/O bus and memory bus affects network performance

Memory bus

Faster, closer: tightly coupled machine.

I/O bus

Slower, farther: cluster.

Example: Like a post office at your front gate (fast) versus one across town (slower).

Network topologies: the big picture

- Many topologies exist; each trades performance against cost
- Commercial machines often mix several topologies, for packaging, cost and available parts
- We study five: Bus, Crossbar, Multistage (Omega), Completely connected, Star

Dynamic

Bus, Crossbar, Omega

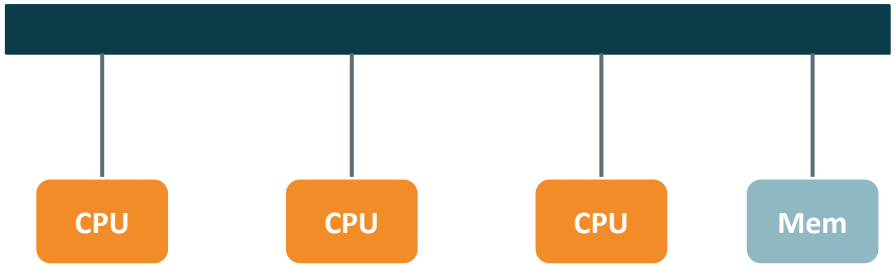
Static

Completely connected, Star

Rule to remember: better performance usually costs more.

Topology 1: Bus

One shared bus



- All processors use one common bus
- Distance between any two nodes is O(1)
- Good for broadcast: one message reaches all
- Bandwidth of the shared bus is the bottleneck
- Limited to dozens of nodes

Example: A classroom with one speaker at a time: fine for 10 students, chaos for 1,000. Real: Sun Enterprise, Intel Pentium shared-bus servers.

Bus with local memory or caches

(a) No local cache

Every access goes over the bus. Bus gets crowded quickly.

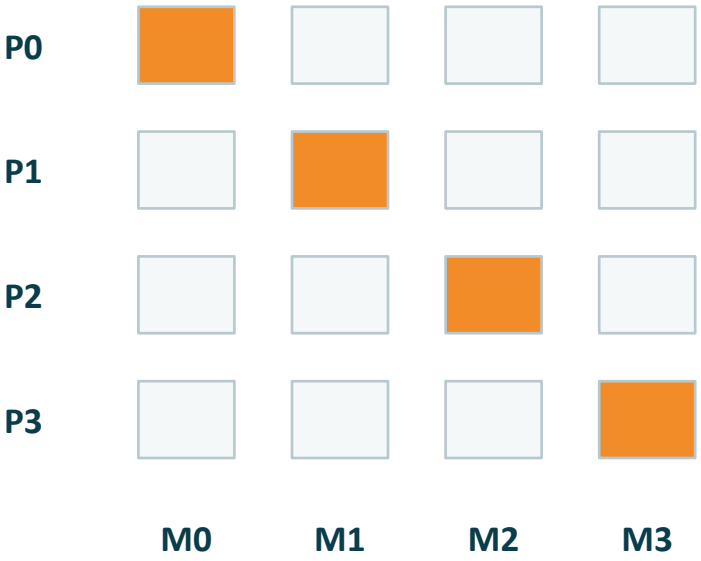
(b) With local memory / cache

Each CPU keeps data close. Fewer bus trips.

Much of the data a processor uses is local to it, so a local memory greatly improves the performance of bus-based machines.

Example: Keeping your notebook on your desk instead of walking to the library every time.

Topology 2: Crossbar



Orange = connections in use

- A p × m grid of switches connects p processors to m memory banks
- Non-blocking: many different pairs talk at the same time
- Cost grows as $O(p^2)$
- Hard to scale to large p

Real machines: Sun Ultra HPC 10000, Fujitsu VPP500.

Why multistage networks exist

Bus

Excellent cost scaling, poor performance scaling.

Crossbar

Excellent performance scaling, poor cost scaling.

Multistage networks strike a compromise between these two extremes.

Example: One narrow road (cheap, jammed) versus a private road for every pair of houses (fast, absurdly costly). Multistage = a sensible grid of roads.

Topology 3: Omega network (multistage)

- Most commonly used multistage network
- p inputs and p outputs
- Made of log p stages (p is a power of 2)
- Built from small 2×2 switches
- Each stage connects the links with a "perfect shuffle"

Size for p = 8

Stages: log 8 = 3
Switches: (8/2) × 3 = 12
Each stage has 4 switches

Example: A crossbar for 8 inputs needs 64 crosspoints; an Omega network needs only 12 small switches.

Perfect shuffle (like shuffling cards)

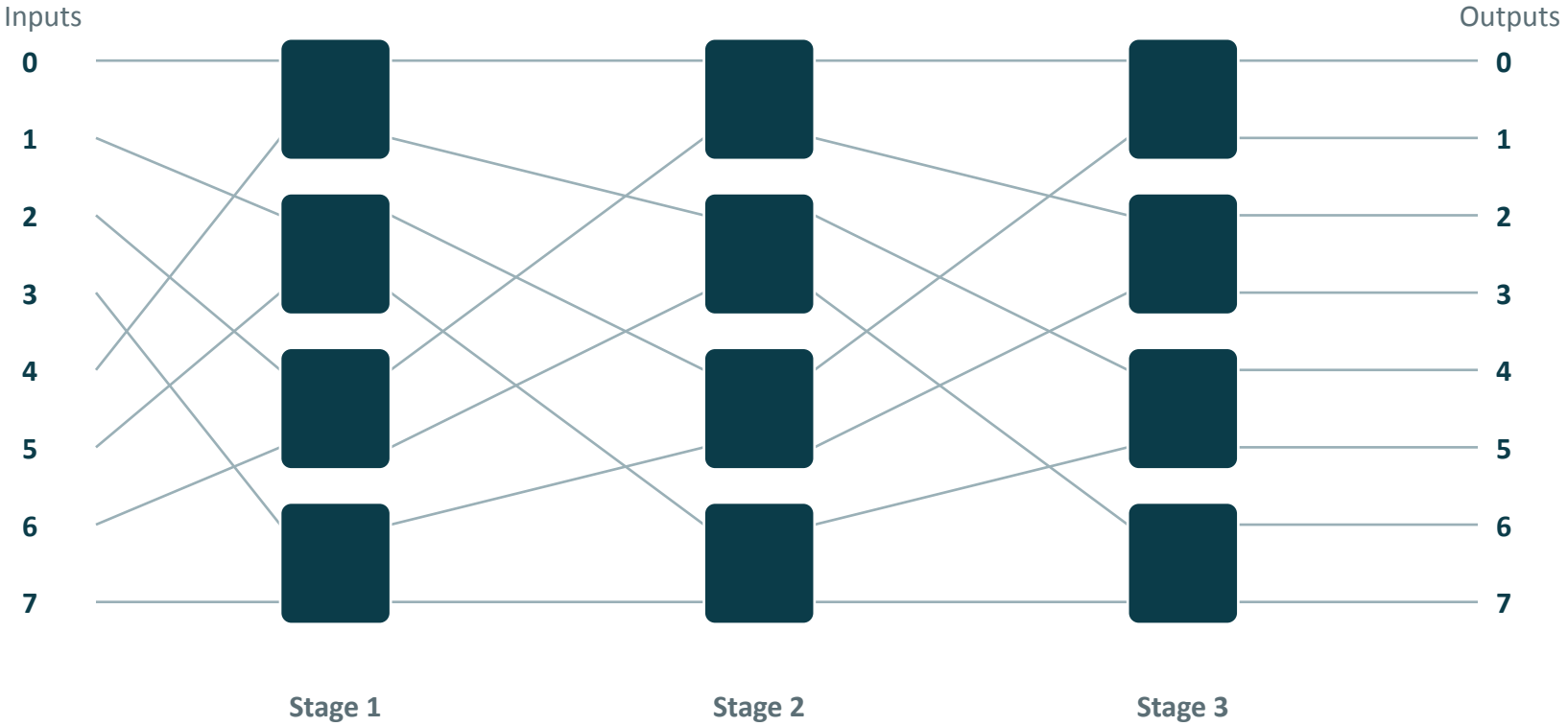
- Split the inputs into two halves and interleave them
- First half go to even outputs, second half to odd outputs

Input i	0	1	2	3	4	5	6	7
Output j	0	2	4	6	1	3	5	7

Rule from the lecture: $j = 2i$ if $i < p/2$, otherwise $j = 2i + 1 - p$.

Example: Split a deck of 8 cards into two piles of 4 and riffle them together: card 4 lands in position 1.

Omega network with 8 inputs and 8 outputs



Omega network: switch modes and routing

Pass-through

Straight: top input to top output, bottom to bottom.

Cross-over

Crossed: top input to bottom output and vice versa.

- Let s = source in binary, d = destination in binary
- At stage k, compare bit k of s and d (starting from the most significant bit)
- Same bits: pass-through. Different bits: cross-over
- Repeat for all log p stages

Example: Send from 010 to 111: bits (0,1) differ: cross. (1,1) same: pass. (0,1) differ: cross.

Omega network can block

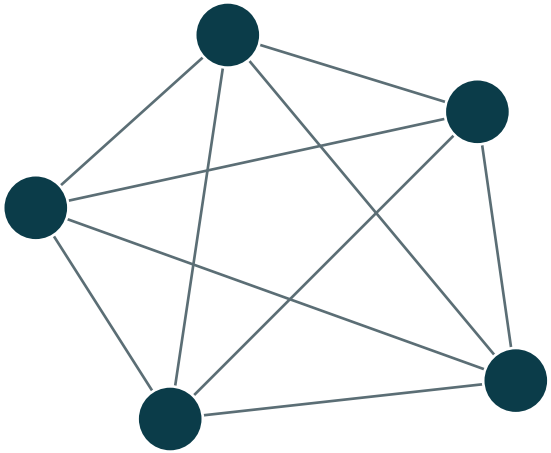
- Two messages may need the same link at the same time
- Lecture example: 010 to 111 and 110 to 100 both need link AB
- One message must wait: the network is not non-blocking
- A crossbar never has this problem, but costs far more

Trade-off

Omega cost: $O(p \log p)$
Crossbar cost: $O(p^2)$
Omega is cheaper but sometimes waits.

Example: Two cars merge into one lane at a junction: one has to wait for the other.

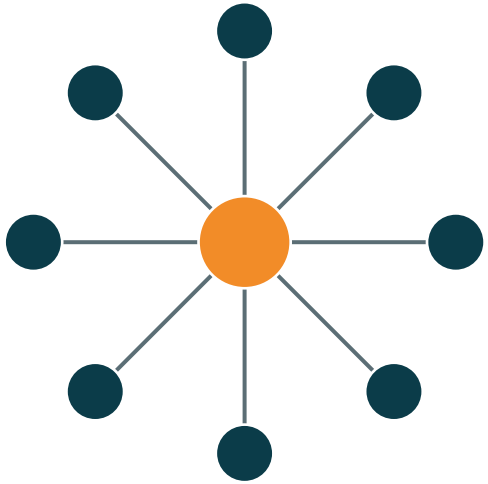
Topology 4: Completely connected network



- Every processor is linked to every other processor
- Number of links grows as $O(p^2)$
- Performance is excellent
- Hardware is impossible for large p
- Static counterpart of the crossbar

Example: Everyone in a class has a direct phone line to everyone else: 8 nodes need 28 links, 100 nodes need 4,950.

Topology 5: Star network



- Every node connects only to one central node
- Distance between any two nodes is O(1) (two hops)
- The central node becomes a bottleneck
- Static counterpart of the bus

Example: A school where every message must pass through the front office: simple, but the office gets swamped.

Comparing all topologies

Topology	Type	Cost	Performance	Analogy
Bus	Dynamic	Low	Poor (shared)	One-lane road
Crossbar	Dynamic	$O(p^2)$	Excellent	Private road for each pair
Omega	Dynamic	O(p log p)	Good, can block	Road grid with junctions
Completely connected	Static	$O(p^2)$ links	Excellent	Phone line to everyone
Star	Static	Low	Poor (centre)	Front office

Pairs: Bus ~ Star (cheap, bottleneck). Crossbar ~ Complete (fast, costly).

Quick review questions

- What is the difference between static and dynamic networks?
- Why does a crossbar not scale well for large p?
- How many stages and switches does an 8-input Omega network have?
- Route a message from 011 to 100 in an Omega network.
- Which topology is the static counterpart of the bus? Of the crossbar?

Answers: 2) cost is $O(p^2)$ 3) 3 stages, 12 switches 5) star, completely connected.

Key takeaways

1. A network shares scarce wires and switches between processors and memory
2. Static = direct links; dynamic = switches in between
3. Bus: cheap but bottleneck. Crossbar: fast but costly
4. Omega: a compromise, O(p log p) cost, but it can block
5. Complete and star are the static twins of crossbar and bus